An Experimental Evaluation of Containerized CI/CD Pipelines with Integrated Prometheus–Grafana Observability

Vinayak Tyagi
Department of Computer Science and Engineering
Manipal University Jaipur
vinayaktyagi.ed@gmail.com

***Abstract*—Containerization, CI/CD automation, and observability tooling are each widely studied on their own, but comparatively few studies measure their *combined* operational effect under a controlled experimental design. This paper builds a reproducible pipeline — Docker, GitHub Actions, Prometheus, and Grafana — around a purpose-built subject application, and experimentally evaluates it against two baselines (Baseline A: no containerization; Baseline B: Docker without CI/CD or monitoring), a third, non-monitored but CI/CD-enabled configuration (Docker+CI/CD) that isolates automation's own cost, and against itself at increasing levels of observability once containerized (Docker only, Docker+Prometheus, Docker+Prometheus+Grafana — the last being the full proposed system). Across five controlled experiments (188 runs total: deployment time and reliability across four configurations, resource utilization and monitoring overhead across three cases, CPU-stress detection, memory-stress/OOM behaviour, and failure detection/recovery), we find that: containerized deployment adds roughly 0.86s of fixed overhead over a raw process start (1.45s vs 0.59s mean); a CI/CD-automated deployment is reliable (7/8 successful triggers) but roughly 35× slower end-to-end (50.2s vs. 1.45s mean) than a pre-built local Docker deploy, because it repays the full build+test cost every run; a Prometheus scrape loop detects induced CPU saturation in 5.65s mean (20/20 runs) and kernel-level container failure in 1.15s mean (20/20 runs), both bounded below by the 2s scrape interval; and, in the original run, the monitored application container's own CPU reading *decreased* as monitoring containers were added (11.0% → 3.8% → 1.8% mean) while throughput dropped (2737 → 2641 → 2449 req/s). We initially read this as host-level CPU contention, but a later replication and a controlled follow-up experiment show the CPU trend is largely a sampling artifact and bound monitoring overhead at about 1.4% of throughput, so the contention explanation is not supported (Section X). All raw per-run measurements and analysis scripts underlying these results are published at https://github.com/vinayaktyagi10/cicd-monitoring-study.**

***Index Terms*—CI/CD, Docker, containerization, DevOps, Prometheus, Grafana, observability, empirical evaluation**

I. Introduction

DevOps practice has converged on three largely independent pillars: *containerization* (packaging an application and its runtime into a portable, isolated unit, typically with Docker), *CI/CD automation* (build, test, and deployment triggered automatically on every change, typically via a hosted runner such as GitHub Actions), and *observability* (continuous collection and visualization of runtime metrics, typically via a Prometheus scrape loop feeding a Grafana dashboard). Each pillar is well studied in isolation: containerization has an extensive performance-overhead literature going back to Felter et al. [1]; CI/CD automation is a standard subject of DevOps tooling papers; and Prometheus/Grafana integration is a common implementation detail in cloud-native systems papers. What is measured far less often is the *combined* operational profile of running all three together — specifically, how much each additional layer costs in deployment time, steady-state resource consumption, and throughput, and what it buys back in faster failure detection.

This paper reports a controlled, repeated-measures experimental evaluation of exactly that combination. We build a small subject application, wrap it in a Docker image, automate its build and test via a GitHub Actions workflow, and instrument it with Prometheus (scraping native application metrics) and Grafana (dashboarding). We then run it under five experiments spanning normal operation, CPU stress, memory stress, and induced failure, comparing it against two baselines — an uncontainerized process (Baseline A) and a containerized, CI/CD-free, unmonitored deployment (Baseline B) — and a third, intermediate configuration (Docker+CI/CD) that isolates the cost of automation alone, before monitoring is added on top to reach the full proposed system. Every run was logged in full, including two environment-specific tooling constraints encountered during the study and how each was resolved (Section X), so the results in Section VIII are traceable back to raw measurements rather than summary claims.

A. Research Questions

- **RQ1.** How does containerization affect application deployment time and resource utilization in a CI/CD environment?
- **RQ2.** How does integrating automated monitoring affect the detection of performance degradation and failures?
- **RQ3.** What is the overhead introduced by Prometheus–Grafana monitoring in a containerized CI/CD deployment?
- **RQ4.** Does automated CI/CD combined with monitoring improve deployment reliability compared with a non-automated baseline?

B. Objectives

O1. Develop a reproducible containerized CI/CD pipeline using GitHub Actions.
O2. Integrate Prometheus and Grafana for real-time infrastructure/application monitoring.
O3. Measure CI/CD performance, container resource utilization, and monitoring overhead.
O4. Evaluate failure detection and recovery under controlled fault conditions.

II. Contributions

1) **A combined, repeated-measures evaluation.** Controlled measurements of deployment time, resource use, throughput, and failure detection and recovery across three configurations (no containerization, Docker only, Docker with CI/CD and Prometheus–Grafana), with 20 runs per configuration (8 for the Docker+CI/CD arm, which used real GitHub Actions runs), reported with means, spreads and confidence intervals from raw per-run data.
2) **A same-hardware replication that separates robust from environment-dependent results.** Experiments 1–5 were repeated with unmodified scripts after the operating system was reinstalled. Table X classifies each finding. Deployment ordering, the cost of the CI/CD build-test cycle, recovery time, peak CPU under stress and the OOM kill with resident memory reproduced. Absolute times, throughput, detection latencies and the zero-filled OOM result did not.
3) **A retracted explanation.** The original run suggested that monitoring degrades throughput through host-level CPU contention. A controlled follow-up (Experiment 6, CPU read from kernel counters over the load window) did not support this: the monitoring containers drew about 0.03 cores, the apparent fall in application CPU was a sampling artifact, and with a stable keep-alive client throughput was within 1.4% of Docker-only. One of two keep-alive configurations detects a loss of that size, so we report a bound rather than zero overhead. We report the original claim as unsupported rather than retaining it.
4) **A kernel-dependent failure mode in memory-stress testing.** On a kernel that reclaims zero-filled transparent hugepages, a memory-stress test that allocates zeroed memory never triggers an OOM kill, while the same test with non-zero data kills the container 20/20 times. Fault-injection results that depend on memory pressure should therefore state the kernel and fill pattern.
5) **A traceable artifact, with its limits stated.** Raw per-run data, experiment and analysis scripts, and a one-command rebuild of the tables, statistics and figures from that raw data (`make reproduce`, no Docker needed) are published with the paper. Re-collecting data is repeatable but not deterministic (Section X).

III. Related Work

Table I summarizes the studies most directly relevant to this work's three pillars. This is a seed set, extended with four further works found by targeted web searches, assembled from a targeted IEEE Xplore search (queries: *Docker container performance overhead benchmark*; *containerized CI/CD pipeline performance evaluation*; *Prometheus Grafana monitoring microservices*) rather than an exhaustive systematic review, and is intended as a starting point for a full literature survey before submission.

Felter et al. [1] remains the most-cited apples-to-apples comparison of native execution, Docker containers, and KVM virtual machines, finding containers add near-zero CPU/memory overhead relative to native but that both containers and VMs pay a cost on I/O-bound workloads. Lingayat et al. [2] and Mavridis and Karatza [3] extend this line to deployment/startup time and to containers nested inside VMs respectively, but both stop at the container layer: neither introduces a CI/CD pipeline or a monitoring stack, so their measurements cannot speak to RQ2–RQ4. Ekanayaka et al. [4] is the closest system to the one built here — *KubFlow* integrates Git, Jenkins, ArgoCD, Terraform, Docker, Kubernetes, Prometheus, and Grafana into a single DevOps infrastructure — but its evaluation is a pass/fail checklist of eight qualitative test scenarios plus subjective colleague ratings, not a quantitative, repeated-measures comparison against a non-monitored or non-automated baseline.

Three further works bear directly on individual pillars. Dinga et al. [5] empirically evaluate the energy and performance overhead of four monitoring tools, Prometheus among them, on Docker-based systems and report that the overhead depends on the tool and on the frequency and workload conditions; this is the closest prior work on monitoring overhead, but it isolates the monitoring tools and does not combine them with a CI/CD pipeline or fault injection. Our Experiment 6, which measures the monitoring containers' CPU from kernel counters, is complementary: it found $\approx$0.03 cores for cAdvisor, Prometheus and Grafana together on this host, which is compatible with a tool-dependent overhead but is a single configuration and not a comparison across tools. Hilton et al. [6] study the usage, costs and benefits of continuous integration across 34,544 open-source GitHub projects, which motivates measuring the cost of the CI step itself, as the Docker+CI/CD arm does. Forsgren, Humble and Kim [7] establish time-to-restore and change-failure rate as standard software-delivery performance measures; our MTTD and MTTR are the container-level analogues, measured in seconds on a single service rather than as organization-level outcomes. Container background is covered by Bernstein [8]. Three further candidates found in the search — an integrated CI/CD-pipeline study [9], a DevOps security-first CI/CD framework [10], and an AWS/Kubernetes Prometheus–Grafana deployment study [11] — have been identified but not reviewed in depth.

TABLE I
RELATED WORK SUMMARY

Paper	CI/CD	Docker	Monitoring	Metrics	Limitation (relative to this study)
Felter et al. 2015 [1]	No	Yes (vs. KVM)	No	CPU, memory, network, disk microbenchmarks; MySQL/Redis throughput	Isolates container vs. VM overhead only; no CI/CD, no monitoring stack, no failure-detection measurement.
Lingayat et al. 2018 [2]	No	Yes (vs. bare metal / VM)	No	Container startup time	Single metric (startup time); no automation pipeline, no monitoring, no fault injection.
Mavridis & Karatza 2017 [3]	No	Yes (nested in VM)	No	Standard container benchmarks	Studies an orthogonal deployment topology (container-in-VM); no CI/CD or monitoring integration.
Ekanayaka et al. 2023 [4]	Yes (Jenkins+Argo CD)	Yes (Kubernetes)	Yes (Prometheus+Grafana)	Pass/fail test scenarios; subjective 1–5 feature ratings	Same three pillars as this study, but evaluation is qualitative/subjective, not a controlled quantitative comparison against a non-monitored or non-automated baseline.
This study	Yes (GitHub Actions)	Yes (Docker Compose)	Yes (Prometheus+Grafana)	Deployment time, reliability, CPU/mem utilization, throughput, latency, MTTD, MTTR — n=20 per configuration where feasible	—

A. Research Gap

Existing studies frequently evaluate CI/CD automation, containerization, or observability independently (Table I, rows 1–3), or combine all three but report only qualitative, pass/fail integration testing rather than a controlled quantitative comparison (row 4). Comparatively few studies provide a controlled empirical evaluation of their *combined* operational impact across deployment performance, resource utilization, and failure detection, measured against explicit non-containerized and non-monitored baselines with repeated trials. This is the gap this study targets.

IV. Proposed Framework

Fig. 1 shows the overall pipeline architecture and data flow. A developer pushes to the Git repository, which triggers a GitHub Actions workflow that runs the test suite and, on success, builds and pushes a Docker image. The image is then run under Docker Compose alongside a cAdvisor container-metrics exporter, a Prometheus server that scrapes both the application's native `/metrics` endpoint and cAdvisor, and a Grafana instance provisioned with Prometheus as its data source.

Fig. 2 details the GitHub Actions workflow itself, exactly as defined in the repository's `ci.yml`: a `test` job (checkout, Python 3.12 setup, dependency install, `pytest`) gates a `docker-build` job (checkout, GHCR login, build and push) via `needs: test` — the build only runs if the tests pass. This is the real workflow triggered for every Docker+CI/CD measurement in Table V, not a simplified stand-in.

Fig. 3 shows the resulting Grafana dashboard under live load: request rate and p95 latency by endpoint, the application's self-reported process CPU% and memory, and the Prometheus `up` target-health indicator, all auto-provisioned (no manual dashboard configuration) and updating in real time from the same Prometheus instance used throughout the experiments.

Subject application. Rather than an existing multi-service demo application, the study uses a small, hand-built FastAPI service with three endpoints designed as clean experimental levers: `/health` (a near-constant-time route used as the load-test and liveness target), `/cpu` (a configurable-iteration pure-Python busy loop used for CPU stress), and `/memory` (a configurable-size heap allocation, optionally retained, used for memory stress). This choice trades away the multi-service realism of a demo application such as Sock Shop for the ability to attribute every measured effect to a specific, known cause rather than to inter-service scheduling or network confounds. The application exposes Prometheus metrics natively via the `prometheus_client` library — a request counter and latency histogram per endpoint, plus two self-reported process gauges, `app_process_cpu_percent` and `app_process_memory_bytes`, sampled every second from `time.process_time()` and `/proc/self/status` deltas respectively — rather than through a side-car exporter.

V. Experimental Setup

Table II lists the experimental environment. All experiments ran on a single physical host (no cloud VM); GitHub Actions hosted runners were used only for the CI/CD build-and-test step itself, so that step reflects a genuinely independent, reproducible environment while deployment, load generation, and monitoring remained under full local control.

A load-testing tool such as `hey` or `wrk` was deliberately not installed; a small standard-library Python script (`loadgen.py`) issues concurrent HTTP requests via

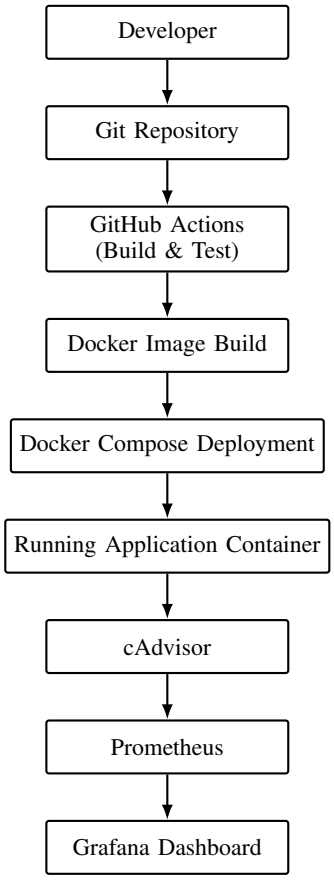


Fig. 1. Overall pipeline architecture: source control triggers automated build and test, which produces the container image deployed alongside the observability stack.

TABLE II
EXPERIMENTAL ENVIRONMENT

Parameter	Value
CPU	AMD Ryzen 7 8845HS (8 cores / 16 threads)
RAM	14 GiB
Host OS	Arch Linux (Omarchy), kernel 7.1.9-arch1-2
Docker	29.7.2 (containerd snapshotter storage driver)
GitHub Actions	`ubuntu-latest` hosted runners
Prometheus	v2.55.1, 2s scrape/evaluation interval
Grafana	11.2.0
cAdvisor	v0.49.1
Application base image	`python:3.12-slim`
Application framework	FastAPI 0.115.0 / Uvicorn 0.30.6
Load generator	Dependency-free Python `ThreadPoolExecutor` + `urllib` script (no external binary installed)

`ThreadPoolExecutor` and records per-request latency, trading raw throughput ceiling for zero external dependencies and full inspectability.

VI. EXPERIMENTAL SCENARIOS

Five experiments were run, summarized in Table III, followed during validation by a sixth that re-measures monitoring overhead under controlled load (Section VI-F). Each is described in more detail in the corresponding results subsection. A dedicated script per experiment (`exp1_deploy.py` through `exp5_failure_recovery.py`) brings the relevant Docker Compose profile up, executes the scenario, records raw per-run measurements to CSV, and tears the stack down before the next run, so runs are independent of one another. An `aggregate.py` script then computes the mean/standard deviation/min/max summaries reported in Section VIII directly from these raw CSV files.

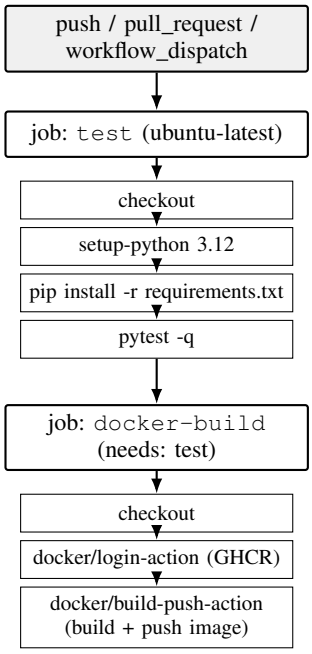


Fig. 2. GitHub Actions workflow (`.github/workflows/ci.yml`). The `docker-build` job only runs if `test` succeeds (`needs: test`); this is the real workflow triggered for the Docker+CI/CD deployment-time measurements in Table V.

TABLE III
EXPERIMENTAL SCENARIOS

#	Scenario	Configurations × runs
1	Deployment time & reliability	manual (20), Docker (20), full pipeline (20), Docker+CI/CD (8)
2	Resource utilization, response time, throughput, monitoring overhead	Docker (20), Docker+Prometheus (20), Docker+Prometheus+Grafana (20)
3	CPU stress detection	20
4	Memory stress / OOM behaviour	20
5	Failure detection & recovery	20
6	Monitoring overhead vs. host CPU vs. throughput (validation)	4 cases (20 each) + pinned arm, two load clients

A. Normal operation and baseline comparison (Experiment 1)

Four configurations are compared, only two of which are baselines proper: *manual* (a raw `uvicorn` process, no container — **Baseline A**) and *Docker* (a locally built and started container, no CI/CD, no monitoring — **Baseline B**). The

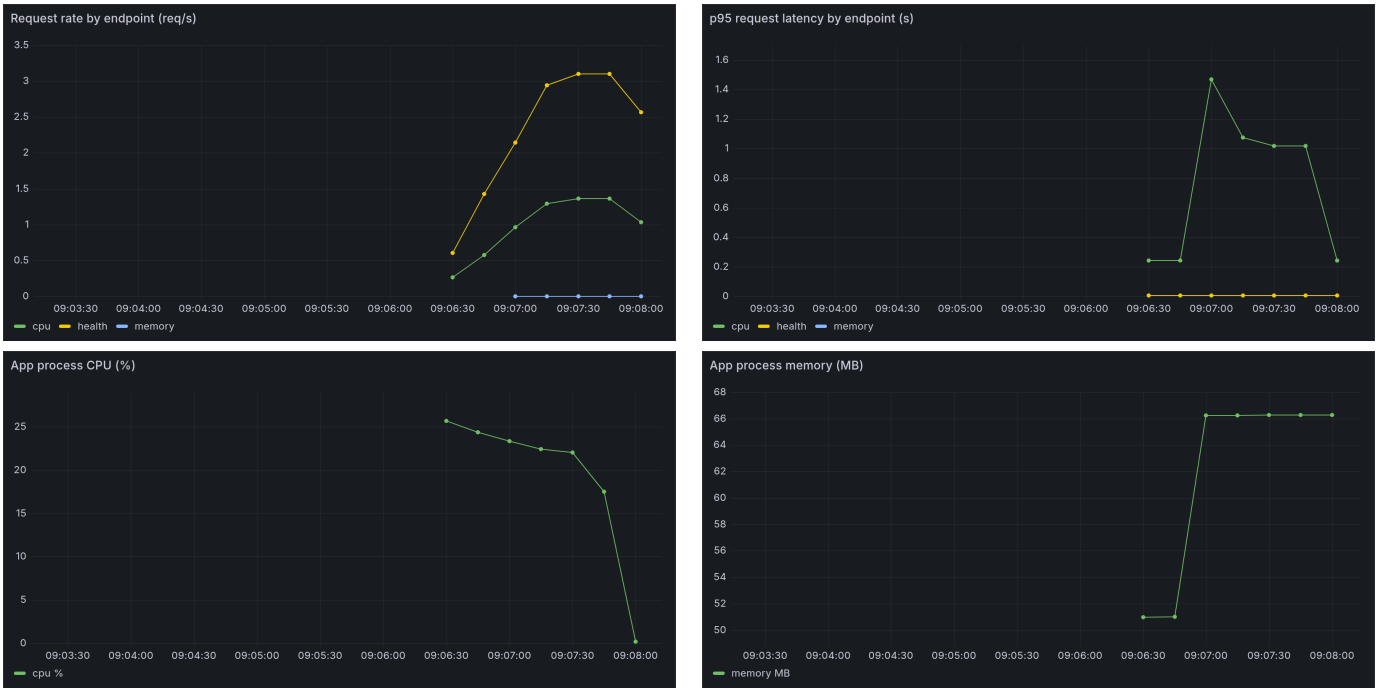


Fig. 3. Grafana dashboard panels under a scripted live workload (last 5 minutes), auto-provisioned against the Prometheus datasource: request rate and p95 latency by endpoint (top), application process CPU and memory (bottom). The target-health panel (not shown) reported `up=1`. Each panel was captured separately from the running stack by `capture_grafana.py`. Unlike Figs. 4–10 these are screenshots of live state and cannot be reproduced pixel for pixel; the Prometheus series behind each panel for this capture are saved in `experiments/raw/grafana_panels.json`.

remaining two are not baselines but an explicit progression toward the proposed system: *Docker+CI/CD* (a real GitHub Actions `workflow_dispatch` run — test job then Docker build/push — timed end-to-end, then deployed) is an **intermediate configuration** that isolates what automation alone costs, with monitoring still absent; and *full pipeline* (Docker Compose bringing up the application alongside cAdvisor, Prometheus, and Grafana) is the **proposed system**, adding observability on top. This ordering — Baseline A → Baseline B → Docker+CI/CD → full pipeline — lets each of containerization, automation, and observability be attributed its own marginal cost rather than only the combined effect of all three at once. For each run, whatever was previously running is torn down, the configuration is started, and wall-clock time to the first successful health check (and, for the full pipeline, to Prometheus and Grafana also responding) is recorded.

B. Resource utilization and monitoring overhead (Experiment 2)

Three cases isolate the marginal cost of each monitoring layer: Docker only, Docker+Prometheus(+cAdvisor), and Docker+Prometheus+Grafana(+cAdvisor). For each of 20 runs per case, a fixed load (300 concurrent requests to `/health`, concurrency 10) is issued while `docker stats` is polled every second for every container in that case, giving per-container mean CPU% and memory alongside the load test's own throughput, mean/p95 latency, and error rate.

C. CPU stress detection (Experiment 3)

Eight threads concurrently flood `/cpu` (an 8-million-iteration pure-Python loop per call) against the Docker+Prometheus case. Detection time is measured as wall-clock time from load start until Prometheus first reports the application's self-reported `app_process_cpu_percent` gauge at or above 60%, polled every 0.5s.

D. Memory stress / OOM behaviour (Experiment 4)

The application container is run directly (not via Compose) with a hard 150 MB memory limit and `--restart on-failure:1`. Repeated `/memory?mb=20&hold=true` calls accumulate retained allocations until the kernel OOM-kills the container; `docker inspect` gives ground-truth `OOMKilled` and `RestartCount`, captured at the moment of observation since Docker resets `State.OOMKilled` once the restart policy brings the container back up.

E. Application failure and recovery (Experiment 5)

Against the Docker+Prometheus case, the application container is killed (`docker kill`) and Prometheus's `up{job="app"}` series is polled every 0.3s until it reports 0 (detection time). The container is then restarted (`docker start`) and both `/health` and `up{job="app"}==1` are polled until both recover (recovery time).

F. Monitoring overhead under controlled load (Experiment 6)

Added during validation because Experiment 2's ≈0.11 s load window is shorter than one `docker stats` sample.

Four cases (Docker, +cAdvisor, +Prometheus, +Grafana) each receive a closed-loop load of 20 s at concurrency 10 against `/health` on a freshly started stack, with case order shuffled within each of 20 blocks. CPU is read from kernel counters at exactly the start and end of the load window: `/proc/stat` for the whole host, each container's cgroup v2 `cpu.stat`, the load generator's own process time, and the `docker-proxy` processes that forward the published port; whatever remains is unattributed background. A CPU-pinned arm places the application, load generator and monitoring containers on disjoint physical cores for the Docker and +Grafana cases. The experiment was run twice with two load clients: the Experiment 2 client, which opens a new TCP connection per request, and a keep-alive client that reuses one HTTP/1.1 connection per worker (`exp6_overhead.py --keepalive`). Raw per-run rows (one per run, with host, per-container and load-generator CPU, throughput and latency) are in `experiments/raw/exp6_overhead*.csv` (four files: both clients, each at one and four load-generator processes); the analysis is `analyze_overhead.py`.

VII. Statistical Analysis

Every interval and test below is computed from the per-run CSV rows, with sample statistics (denominator $n-1$) and a two-sided $\alpha=0.05$, by `stats_lib.py` (unit-tested against `scipy`) and `validate.py`. Table IV lists each method with the reason for choosing it and its main limit. We report intervals and effect sizes first and p-values second: with 20 runs per cell a p-value alone says little about whether a difference matters. The Experiment 2 comparisons in Section VIII quote Cohen's d (`statistical_analysis.py`); everywhere else we use Hedges' g, which differs only by a small-sample correction (under 3% at $n=20$ per group).

Checking the assumptions. We tested the 30 original-versus-replication metrics of Table IX (including the non-zero-fill Experiment 4 rows). A Shapiro–Wilk test rejects normality in at least one of the two groups for 22 of 30 metrics, so the t-based intervals are approximate for most of them. Welch's test and Mann–Whitney agree on 27 of 30 metrics at $\alpha=0.05$, which is why we report both. The disagreements are informative. MTTR has equal means (0.7% apart; Welch $p=0.52$) but a rank test separates the distributions (Cliff's $\delta=-0.80$, Holm-adjusted $p=0.0002$): the replication's recovery times are tighter, a difference in spread that is practically negligible for a metric near 2.0 s but is not "identical". Prometheus CPU under +Grafana and the non-zero-fill time to OOM also split the two tests.

Multiple comparisons. The inline tests in the Results (the Experiment 2 pairwise comparisons and the Experiment 6 contrasts) are not corrected, and we say so where they appear. For the replication comparison we also computed Holm-adjusted p-values across all 30 metrics: 20 of 22 rows marked different keep $p < 0.05$ after the correction. The two that do not are the non-zero-fill OOM rows. There the interval rule says different, while Welch's test finds the two groups indistinguishable ($p\approx0.33$), because every replication run is nearly identical (5 requests, 11.06 s), so the rule's interval collapses to a point. The interval rule and Welch's test agree on 28 of 30 metrics, and the two they split on are exactly these. We keep the rule as stated, since it was fixed in advance, and read those two rows as reproduced in Table X.

What we did not do. We did not power the experiments for small effects: the Experiment 6 keep-alive contrasts have 10–20 runs per case, so differences near 1% are at the edge of detection. We analyzed Experiment 6 as independent groups although its runs come in randomized blocks, and a blocked or paired analysis would likely be more sensitive. We did not model dependence between sequential runs, which matters because throughput tracked host load in the per-request client. We did not run a formal equivalence test, so "consistent" means not distinguishable, not shown equal.

VIII. Results

A. Deployment time and reliability (RQ1, RQ4)

Table V reports wall-clock deployment time for the three fast configurations and CI/CD-inclusive deployment time for the fourth. Containerizing a fixed local image adds a consistent ≈0.86s of fixed overhead over a raw process start (1.454s vs. 0.592s mean); adding the full monitoring stack on top of Docker adds a further ≈0.10s (1.552s vs. 1.454s mean) since Compose brings up four containers rather than one but all four start in parallel. The CI/CD-automated path is roughly 35× slower (50.2s vs. 1.454s mean) because it re-pays the full test-then-build cost on every deployment rather than reusing a pre-built image — this is the direct cost of automation, traded against the reliability and repeatability of not requiring a human to run the equivalent commands by hand. Of 8 triggered CI/CD runs, 7 completed successfully; the one failure was traced to a transient delay in the workflow being registered by GitHub's API immediately after being triggered, rather than to a defect in the pipeline itself, and is reported as observed rather than discarded.

Fig. 4 visualizes Table V on a log scale, since the CI/CD configuration is roughly 30–85× slower than the other three and would otherwise flatten them to indistinguishable bars.

B. Resource utilization, throughput, and monitoring overhead (RQ1, RQ3)

Table VI reports the application container's own resource use and the load test's response characteristics; Table VII isolates the monitoring stack's own resource draw. Zero errors were observed across all 60 runs (180×300 = 18,000 requests). Point estimates for throughput fall as monitoring is added (2737 → 2641 → 2449 req/s, Docker → +Prometheus → +Grafana); a Welch's t-test (unequal variances, $n=20$ per case, computed directly from the raw per-run CSV rather than from the summary means) confirms the two endpoints of this trend are statistically distinguishable (Docker vs. +Grafana: $t=3.57$, $p=0.0011$, Cohen's $d=1.13$), but the middle step is not (Docker vs. +Prometheus: $t=1.37$, $p=0.177$) — the overall direction is real, but the drop is not uniform run-to-run between every adjacent pair. Mean and p95 latency both

TABLE IV
STATISTICAL METHODS, WHY EACH WAS CHOSEN, AND ITS MAIN LIMIT.

Quantity	Method	Why this method	Main limit
Mean of a metric	t-interval, $n-1$ d.f.	Small samples (20 runs per cell, 8 for CI/CD) with unknown variance	Assumes roughly normal means; see the normality check below
Difference between two cases	Welch's t-test with its interval	Variances differ a lot between cases (MTTR std 0.006 s against MTTD std 3.7 s), so equal-variance Student's t would be wrong	Same normality assumption; runs treated as independent
Robustness of that test	Mann–Whitney U, Cliff's δ	Rank-based, no normality assumption, insensitive to outliers; δ is the probability one group exceeds the other minus the reverse	Tests a shift in distribution, not in the mean; ties inflate δ for near-constant metrics
Standardized effect size	Hedges' g with a normal-approximation interval	Puts metrics on one scale; the small-sample correction matters at n=10–20	The interval is approximate and rough at n=10
Success rates (7/8, 20/20)	Wilson score interval	Stays inside $[0, 1]$ and behaves at small n and at 0 or 1, where the Wald interval fails	Describes the binomial rate only
Throughput against CPU	Pearson r with a Fisher-z interval	Measures how strongly run-level throughput tracks host load	Assumes linearity; sequential runs may be autocorrelated
Pinning (contention) test	Difference-in-differences with a percentile bootstrap (10,000 resamples, fixed seed)	A four-group contrast with no closed-form interval; the bootstrap assumes no distribution	Unreliable tails at 10–20 runs per group
Replication verdict	Original mean inside the replication's 95% interval	Fixed before any replication data existed, so it could not be tuned to the result	Collapses when the replication has almost no variance

TABLE V
DEPLOYMENT TIME BY CONFIGURATION (SECONDS)

Config	Mean	Std	Min	Max	N (fail)
Manual (Baseline A)	0.592	0.020	0.577	0.633	20 (0)
Docker (Baseline B)	1.454	0.107	1.267	1.581	20 (0)
Docker+CI/CD	50.231	3.817	44.404	55.362	8 (1)
Full pipeline	1.552	0.031	1.485	1.636	20 (0)

TABLE VI
APPLICATION RESOURCE USE AND RESPONSE CHARACTERISTICS

Config	App CPU%	App Mem (MB)	p95 (ms)	Throughput (req/s)
Docker	11.02 (3.23)	36.11 (0.41)	5.23 (1.00)	2737 (206)
+Prometheus	3.76 (4.42)	35.68 (0.43)	5.34 (0.80)	2641 (226)
+Grafana	1.79 (3.11)	35.79 (0.96)	5.51 (0.86)	2449 (285)

Mean (std. dev.), $n = 20$ runs of 300 requests each per row.

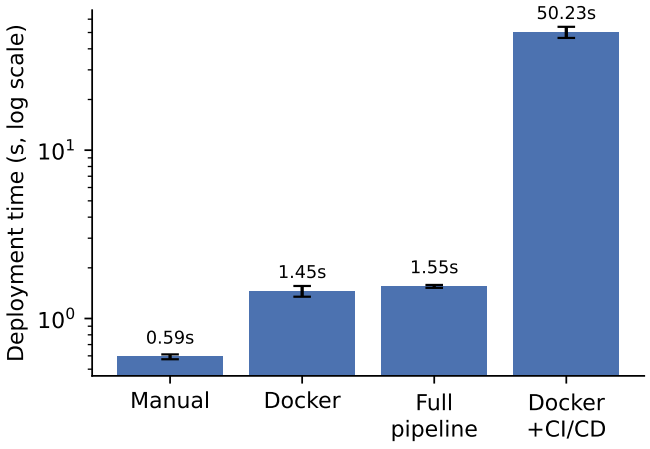


Fig. 4. Deployment time by configuration (log scale; error bars are one standard deviation). Docker adds a small fixed cost over the raw process; the full monitoring stack adds almost nothing further on top of Docker; CI/CD dominates because it re-runs the full build-and-test cycle rather than reusing a pre-built image.

rise numerically but *none* of the three pairwise differences reach significance at α=0.05 (all $p \geq 0.36$); this claim is therefore stated only as a directional observation, not a confirmed effect, and should not be read as strong evidence on its own. Counter-intuitively, the application container's own mean CPU% *falls* over the same progression (11.0% $\rightarrow$ 3.8% $\rightarrow$ 1.8%), and here the statistical evidence is strong: Docker vs. +Prometheus is highly significant (t=5.78, p<0.0001, d=1.83, a large effect), though +Prometheus vs. +Grafana is not (p=0.12). We first read this as monitoring containers competing with the application for cores, but that reading is retracted (Section VIII-H): the load window is only ≈0.11s and the `docker stats` reading covers a longer interval, so many runs read as idle (0/20, 12/20 and 16/20 runs below 1% CPU for Docker, +Prometheus and +Grafana) while non-idle runs read ≈8–11% in every case, so the fall in the mean mostly counts idle readings. The `prometheus_client` counters/histograms/gauges add negligible per-request work, as expected.

Table VII is visualized in Fig. 10, alongside the failure-detection and MTTD/MTTR figures at the end of this section.

C. CPU stress detection (RQ2)

All 20 induced CPU-stress runs were detected by Prometheus, with mean time to detect (MTTD) 5.65s (std 3.68s, min 1.51s, max 18.58s) and mean peak self-reported CPU utilization 96.6% (std 10.0, effectively pegged at one

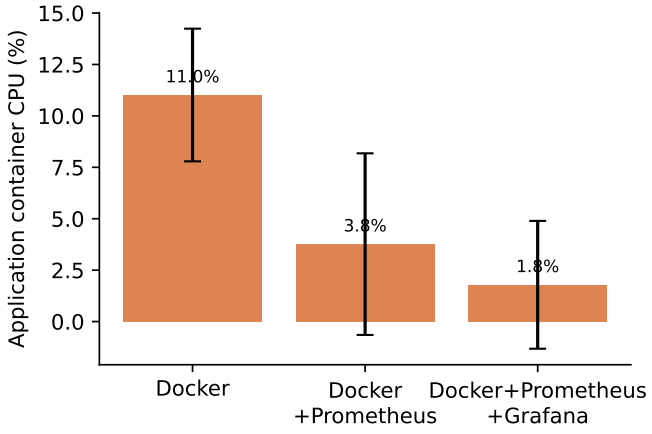


Fig. 5. Application container CPU utilization across the three monitoring cases (Table VI, column 1). CPU% *falls* as more monitoring containers are added — the counter-intuitive result discussed under RQ3.

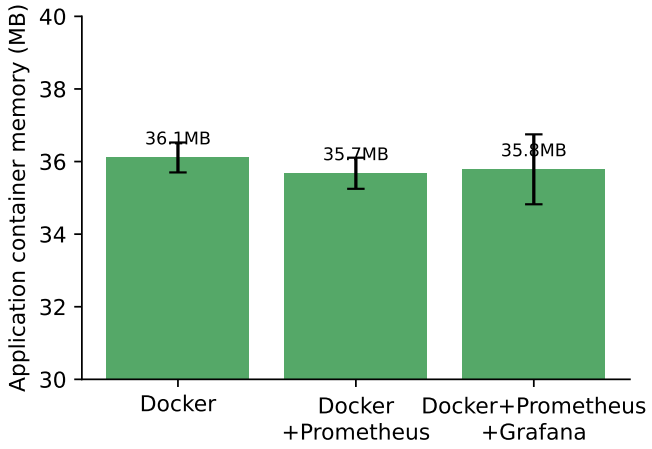


Fig. 6. Application container memory across the three monitoring cases (Table VI, column 2; note the narrow y-axis range). Memory is essentially flat at ≈36 MB regardless of how much monitoring runs alongside the application — unlike CPU, memory is not shared/contended the same way across containers on this host.

full core, consistent with CPython's GIL limiting a single process to ≈100% of one core regardless of thread count). The detection floor is set by the 2s Prometheus scrape interval combined with the 0.5s poll granularity used to observe it; the long right tail (up to 18.58s) reflects run-to-run variance in how quickly the OS scheduler favoured the eight stress threads over the sampling thread, not a monitoring-pipeline delay.

An initial design used mean `/health` latency as the detection signal instead of process CPU%; it was rejected after finding it barely moved under load, because CPython's GIL round-robins between threads at a short, fixed interval (≈5ms) regardless of how CPU-bound the competing work is, so a trivial handler's queueing delay stays small even under heavy contention.

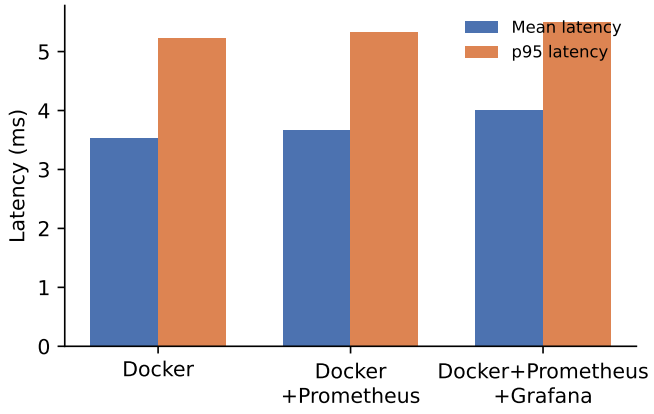


Fig. 7. Mean and p95 request latency across the three monitoring cases (Table VI, columns 3-4). Both rise slightly as monitoring is added, but none of the pairwise differences is significant (Section VIII), so this is a directional observation only.

TABLE VII
MONITORING STACK'S OWN RESOURCE DRAW (TABLE 5: OVERHEAD)

Container	**+Prometheus case**		**+Grafana case**	
	CPU%	Mem (MB)	CPU%	Mem (MB)
cAdvisor	6.13	47.16	6.40	45.39
Prometheus	0.78	59.40	0.80	52.78
Grafana	—	—	0.06	60.70

Mean, $n = 20$ runs per column.

D. Memory stress and OOM behaviour (RQ2)

All 20 runs triggered a genuine kernel OOM-kill and automatic restart (`RestartCount=1`, confirmed via `docker inspect`), after a mean 5.25 successful allocation requests (std 1.09) and a mean peak reported memory of 135.54 MB (std 0.17) against the 150 MB hard limit. Mean time-to-OOM was 11.58s (std 2.23s); this figure is dominated by the fixed per-iteration overhead of the polling loop itself (a `curl`-equivalent request, a `docker inspect` call, and a `docker stats` call per iteration) rather than by the kernel's actual OOM response latency, and should not be read as a monitoring detection time in the same sense as Experiments 3, 5, and 6.

E. Why the zero-filled allocation survives: kernel dependence (Experiment 4)

Observation. The unmodified Experiment 4 script held 20 MB blocks of zero-filled memory in a container limited to 150 MB. Originally every run was OOM-killed after about five requests. On the replication kernel none were: 0/20 runs were killed even after 16 requests (≈320 MB held against a 150 MB limit), and the cgroup's `memory.events` showed the limit reached without an OOM kill. Writing one non-zero byte per page (`/memory?fill=true`) restored the original behaviour: 20/20 kills after a mean of 5.0 requests.

Mechanism. `bytearray(n)` writes its zeros, so the memory is resident (an untouched mapping adds about 1 MB; a 200 MB `bytearray` adds 200 MB even with huge pages

TABLE VIII
EXPERIMENT 4 DIAGNOSTIC: ALLOCATION PATTERN AGAINST THP POLICY FOR THE PROBE PROCESS (5 RUNS PER ROW; 20 BLOCKS OF 20 MB AT MOST, 150 MB LIMIT). *Underused-THP splits* IS THE HOST'S THP_UNDERUSED_SPLIT_PAGE COUNTER DELTA PER RUN.

Allocation	THP for the probe	OOM-killed	Blocks reached (me
Zero-filled	default	0/5	
Zero-filled	disabled	5/5	
Non-zero	default	5/5	
Non-zero	disabled	5/5	

disabled). With transparent huge pages (THP) set to `always`, those writes are backed by 2 MB pages. Recent kernels can split a huge page that is mostly zero and free its zero subpages (the "underused page" shrinker, enabled here: `shrink_underused=1`). When the cgroup reaches its limit the kernel reclaims memory that way instead of killing the process, so an allocation that is only zeros never exhausts the limit. A non-zero byte in every 4 KiB page makes each huge page non-empty, so nothing can be freed and the OOM killer runs.

Evidence. A standalone probe (`diag_thp.py`; same 150 MB, no-swap limit; up to 20 blocks of 20 MB; 5 runs per cell) crosses the allocation pattern with the THP policy, disabling THP for the probe process alone via `prctl` (Table VIII). Only the zero-filled allocation under default THP survives, and it is the only cell where the kernel's underused-huge-page split counter moves (about 170 splits per run). Disabling THP for that one process, with the host, kernel, container and limit unchanged, is enough to restore the OOM kill.

What this means. The OOM-kill result is conditional on the memory being genuinely resident and non-reclaimable. A memory-stress test that holds only zeros is not a stress test on a host with THP enabled and this shrinker present, and a fault-injection study that depends on memory pressure should report the kernel and the fill pattern. The original finding stands for non-zero memory; for the zero-filled variant it is a property of the kernel configuration.

What is not established. We did not record the original host's THP setting and that installation no longer exists, so we cannot show that THP policy is what differed between the original and replication runs. A distribution default of `madvise`, under which a process that does not request huge pages behaves like the "disabled" rows, would be consistent with the original result, but we did not test it. We showed that process-level THP is sufficient to flip the outcome here; we did not toggle the host-wide `shrink_underused` setting (it needs root), and we did not test other kernel versions.

F. Failure detection and recovery (RQ2, RQ4)

All 20 induced container failures (`docker kill`) were detected by Prometheus's `up{job="app"}` series, mean time to detect (MTTD) 1.15s (std 0.21s, min 0.82s, max 2.03s) — close to the theoretical floor set by the 2s scrape interval

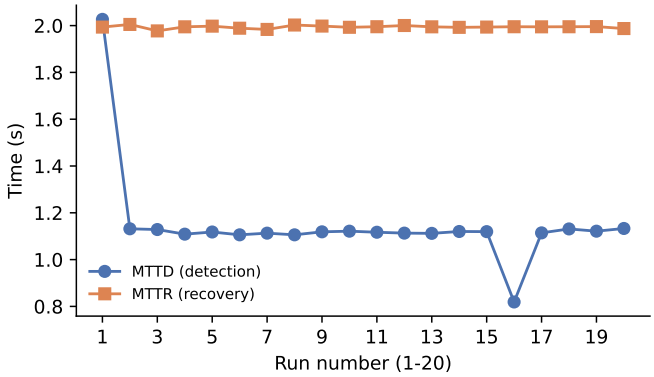


Fig. 8. MTTD and MTTR across all 20 failure-injection runs. MTTD (blue) is consistently faster and tighter than MTTR (orange) across every run, and both are stable run-to-run — neither shows a trend over the sequence of 20 trials, supporting the claim that these are steady-state properties of the pipeline rather than artifacts of run order.

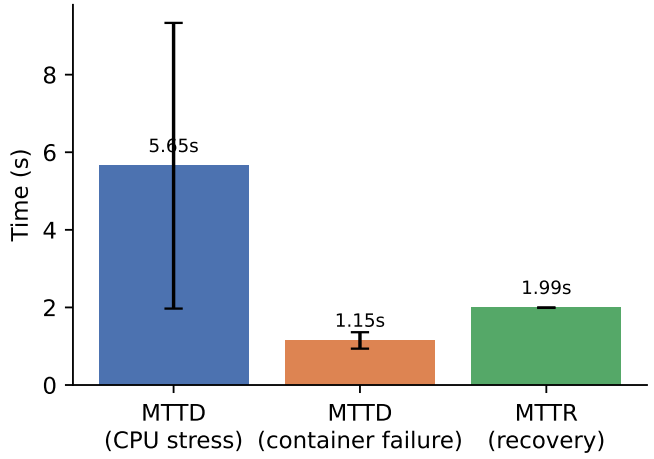


Fig. 9. Mean time to detect (MTTD) a CPU-stress event, MTTD for a killed container, and mean time to recover (MTTR) after restart, with error bars (one standard deviation), computed from the same raw detection/recovery times reported in Section VIII. Binary failure (container killed) is detected roughly 5× faster than gradual CPU saturation, since a scrape either observes the target down or it doesn't, while a saturation threshold must be crossed by an accumulating signal.

combined with 0.3s poll granularity and Docker's own signal-delivery latency. Mean time to recover (MTTR) — container restart to health-check success — was consistently ≈2.0s (std 0.006s, i.e. effectively deterministic), which is essentially Uvicorn's own process startup time inside the container: MTTR here is dominated by application boot, not by any property of the monitoring stack. From this point on, MTTD and MTTR are used throughout in place of "detection time" and "recovery time," consistent with the CPU-stress MTTD reported above and with Figs. 8–9.

G. Monitoring overhead under controlled load (Experiment 6)

Experiment 6 gave two different answers depending on the load client, and the difference is itself a finding. With the

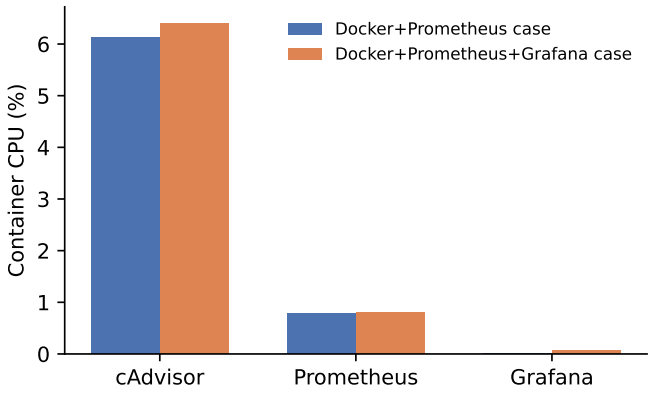


Fig. 10. Monitoring stack containers' own CPU draw (Table VII). cAdvisor dominates the monitoring stack's own overhead at ≈6% CPU in both cases; Prometheus and Grafana are comparatively cheap (<1% each). This is the overhead the monitoring *tooling* itself pays; the follow-up measurement (Section X) found the monitoring containers drew only ≈0.03 cores in total.

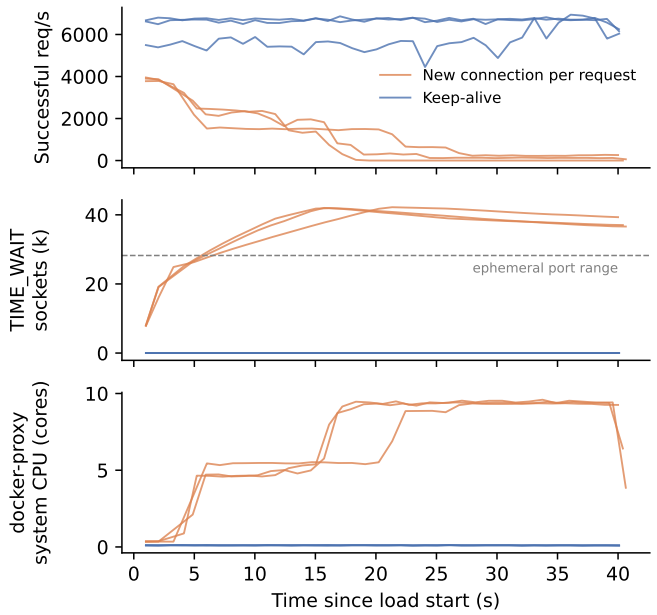


Fig. 11. Diagnostic for the per-request client (3 repetitions per client): throughput collapses as TIME_WAIT sockets pass the ephemeral port range and docker-proxy system CPU rises; the keep-alive client is flat.

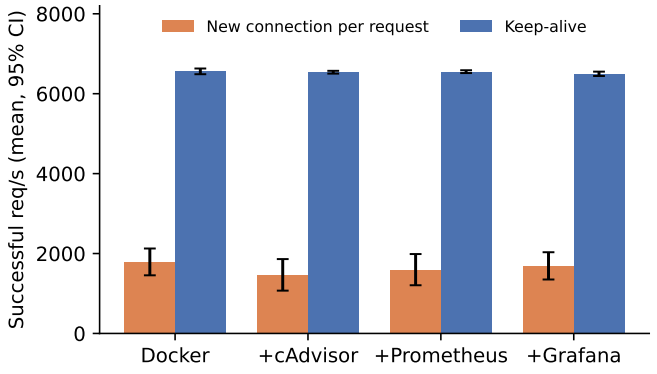


Fig. 12. Experiment 6 throughput per case for both clients (mean, 95% CI, unpinned runs, one load-generator process). The per-request client is low and variable because of TIME_WAIT exhaustion; the keep-alive client shows no monitored case more than 0.9% from Docker-only.

Experiment 2 client, which opens a new TCP connection per request, throughput ranged from 92–2,492 req/s and 25 of 120 runs below 500 req/s, with within-case variation of at least 40%. Throughput tracked host load ($r = -0.98$ with host busy cores) and, more tightly, the CPU time of the docker-proxy processes that forward the published port ($r = -0.991$, 95% CI $[-0.994, -0.986]$); docker-proxy used 4.8–9.6 cores in these runs. A separate diagnostic (3 repetitions per client, 40 s each) explains why. The host has 28,232 ephemeral ports; the per-request client reached 42,205 TIME_WAIT sockets, throughput fell from 3,501 req/s in the first 5 s to 120 req/s after 30 s, and docker-proxy spent 9.1 cores of system time against 0.04 of user time (Fig. 11). With keep-alive connections the same host sustained 6,350 req/s throughout with at most 20 TIME_WAIT sockets and docker-proxy at 0.11 cores. The per-request client therefore mostly measures socket exhaustion on the host over a 20 s load, not the application or the monitoring stack, so its Experiment 6 throughput cannot be attributed to monitoring. Experiment 2's 300-request burst is too short to exhaust the ports, so this does not explain its throughput trend; that trend did not reproduce in the replication (Section VIII-I), and the keep-alive result below bounds the effect at about 1.4%.

The keep-alive client is stable (within-case variation at most 2.3%) and is the one we use to bound monitoring overhead. Against Docker alone, the full stack (+Grafana) differed by -62 req/s, 95% CI $[-149, +25]$ (-0.9%), Hedges' $g = -0.45$ $[-1.07, +0.18]$, and no monitored case moved throughput by more than 0.9% (Fig. 12). The monitoring containers together used 0.032 cores for cAdvisor, Prometheus and Grafana together, with the application container at 1.03–1.04 cores. The CPU-pinned arm gave a difference-in-differences $+8$ req/s, 95% CI $[-94, +105]$, so it neither supports nor excludes a contention effect of that size. One configuration does detect a difference: against Docker alone, +Grafana with four load-generator processes (-1.4%, 95% CI $[-159, -36]$ req/s) is lower, and the interval excludes zero (10 runs per case). We did not correct for the number of contrasts examined, and the 20-run and 10-run designs were not powered for effects this small. We therefore report a bound, not an absence: on this host, with a client that does not exhaust ports, monitoring costs at most about 1.4% of throughput, and one of the two keep-alive configurations detects a loss of that size. We did not identify its mechanism.

H. Revised explanation of the CPU-contention finding

Original claim. In the original run the application's CPU reading fell (11.0% → 3.8% → 1.8%) and throughput fell (2737 → 2641 → 2449 req/s, -10.5%) as monitoring containers were added. We explained both by host-level CPU

contention: the monitoring stack competes with the application for cores, so the application is scheduled less. We no longer hold this as a supported conclusion.

What contention predicts, and what Experiment 6 found.

1) *The monitoring containers should use meaningful CPU.* They did not: about 0.03 cores in total, measured from kernel counters.
2) *Throughput should fall when monitoring is added, with a stable client.* It fell by at most 0.9% with one load-generator process and by 1.4% with four, against the 10.5% originally read.
3) *Moving the monitoring containers to cores the application does not use should remove the drop.* The pinning test is inconclusive in both clients: a difference-in-differences of $+376$ req/s, 95% CI $[-343, +1061]$ with the per-request client and $+8$ req/s, 95% CI $[-94, +105]$ with the keep-alive client. It neither supports contention nor excludes an effect of that size.
4) *Runs with more monitoring CPU should have lower throughput.* With the stable client they do not: across all runs $r = -0.13$, 95% CI $[-0.34, +0.09]$, and within a case $r = -0.07$, 95% CI $[-0.28, +0.16]$. With four load-generator processes the pooled association is $r = -0.63$, 95% CI $[-0.84, -0.25]$, but within a case it is $r = -0.16$, 95% CI $[-0.58, +0.31]$: it comes from the Docker-only against monitored contrast, not from the amount of monitoring CPU.

Revised explanation. (i) The fall in application CPU% is a measurement artifact of a 0.11 s load window sampled by a longer interval (Section VIII), not a scheduling effect. (ii) The original throughput drop cannot be attributed to monitoring. It did not reproduce (replication means of 4041, 3996 and 4033 req/s have no ordering), and with a client that does not exhaust ports the monitoring effect is at most 1.4%, several times smaller. What did drive throughput in the per-request client was socket exhaustion and host load (Section VIII-G). (iii) What remains is a loss of about 1.4% in one of two keep-alive configurations, associated with about 0.03 cores of monitoring CPU. We did not identify its mechanism; contention is one candidate that we have not tested and that the evidence above does not favour over run-to-run variation.

Status. Retracted as a supported conclusion. Experiment 6 excludes large contention effects on this host; it does not exclude effects of about 1% or smaller, and its pinning test is too imprecise to say whether any such effect is contention.

I. Same-hardware replication

Experiments 1–5 were repeated with the unmodified scripts on the same hardware on a later date (20 runs per configuration), but on a freshly installed operating system (CachyOS, kernel 7.2.9, Docker 29.8.2) in place of the Arch/Omarchy 7.1.9 installation of Table II, so this tests robustness to the software environment rather than to run-to-run noise alone. Results were written to a separate directory so the data behind Tables III–VI is untouched. Table IX gives the original and replication means with 95% confidence intervals (`validate.py`); a metric is marked *consistent* only if the original mean lies inside the replication's interval. Most absolute values moved well outside the original intervals, while the relative structure that the research questions rely on held for deployment ordering (including the Docker+CI/CD arm, 8 real GitHub Actions runs, which replicated within its original interval), MTTR and peak CPU under stress (Section X lists what did not reproduce and why). The original run remains the paper's reported dataset because it was the pre-planned measurement and every table is derived from it; the replication is reported as a robustness check, not as a replacement.

IX. Discussion

Taken together, these results answer the four research questions as follows.

RQ1 (containerization's effect on deployment time and resource use). Containerization has a small, consistent fixed cost at deployment time (≈0.86s here) and a resource footprint dominated by the application's own working set (36 MB) rather than by container overhead per se — consistent with the classic finding in Felter et al. [1] that containers add near-zero CPU/memory overhead relative to native execution. The more consequential RQ1 finding in this study is that a CI/CD-automated deployment is not simply "Docker plus a trigger": it is roughly 35× slower end-to-end because it re-runs the full build-and-test cycle, a cost invisible to studies (such as Lingayat et al. [2]) that measure only container startup time in isolation.

RQ2 (monitoring's effect on detecting degradation and failure). Prometheus reliably (20/20 in both experiments) detects both an induced CPU saturation event (mean 5.65s) and an induced process failure (mean 1.15s), with detection latency in both cases bounded below by the 2s scrape interval. The roughly 5× difference between them (5.65s vs. 1.15s) is itself informative: a binary up/down signal is detected almost as fast as the scrape loop allows, while a gradual resource-saturation signal takes longer because it must accumulate enough samples above threshold amid scheduling noise — a distinction a purely qualitative evaluation such as Ekanayaka et al.'s pass/fail test scenarios [4] cannot surface.

RQ3 (monitoring overhead). The original run suggested a structurally unusual overhead: monitoring containers competing with the application for cores, cutting throughput by 10.5% while the application's own CPU% fell. Experiment 6 retracts that explanation (Section VIII-H). The CPU% fall was a measurement artifact; the monitoring containers drew about 0.03 cores; and with a stable client throughput was within 1.4% of Docker-only, a loss one of two configurations detects and whose mechanism is unidentified. Our answer to RQ3 is therefore a bound: on this host the Prometheus–Grafana stack costs at most about 1.4% of throughput and about 0.03 cores, and cAdvisor dominates the tooling's own CPU draw in the original run but not in the replication (6.1% against 2.4%).

RQ4 (does automated CI/CD plus monitoring improve reliability). The evidence here is mixed and should be stated honestly: CI/CD automation was reliable (7/8, 87.5%) but

TABLE IX

ORIGINAL VS. SAME-HARDWARE REPLICATION ON A REINSTALLED OS (20 RUNS EACH). INTERVALS ARE t-BASED 95% CIS ON THE MEAN (SAMPLE SD); Δ% IS REPLICATION RELATIVE TO ORIGINAL. THE EXPERIMENT 4 ROWS ARE FROM THE UNMODIFIED ZERO-FILLED SCRIPT (0/20 OOM KILLS, SEE SECTION X); THE NON-ZERO-FILL ROWS COME FROM THE VARIANT THAT KILLS THE CONTAINER. A ROW IS *consistent* ONLY IF THE ORIGINAL MEAN LIES INSIDE THE REPLICATION'S INTERVAL. THIS RULE IS STRICT WHEN THE REPLICATION RUNS ARE NEARLY IDENTICAL (5 REQUESTS AND 11.06 S IN EVERY NON-ZERO-FILL RUN), SO THOSE ROWS READ *different* ALTHOUGH THE VALUES ARE WITHIN ONE REQUEST, 1 MB AND 0.5 S OF THE ORIGINAL. THE CPU% ROWS MARKED CONSISTENT REST ON THE BIMODAL MEASUREMENT DISCUSSED IN SECTION X. EXPERIMENT 6 HAS NO ORIGINAL RUN AND IS NOT IN THIS TABLE. GENERATED FROM `RESULTS/ORIGINAL_VS_REPLICATION.CSV`.

Metric	Original mean [95% CI]	Replication mean [95% CI]	Δ%	Verdict
Deploy time, manual (s)	0.59 [0.58, 0.60]	0.42 [0.41, 0.43]	-28.9	different
Deploy time, Docker (s)	1.45 [1.40, 1.50]	1.05 [1.04, 1.06]	-27.7	different
Deploy time, full pipeline (s)	1.55 [1.54, 1.57]	1.37 [1.31, 1.43]	-11.8	different
Deploy time, Docker+CI/CD (s)	50.23 [46.42, 54.04]	48.60 [44.05, 53.15]	-3.2	consistent
Throughput, Docker (req/s)	2737.07 [2638.29, 2835.84]	4040.66 [3984.28, 4097.04]	$+47.6$	different
Throughput, +Prometheus (req/s)	2640.65 [2532.08, 2749.23]	3995.78 [3944.85, 4046.70]	$+51.3$	different
Throughput, +Grafana (req/s)	2448.85 [2311.85, 2585.84]	4033.22 [3995.02, 4071.43]	$+64.7$	different
p95 latency, Docker (ms)	5.23 [4.75, 5.70]	3.16 [2.96, 3.36]	-39.5	different
p95 latency, +Prometheus (ms)	5.34 [4.95, 5.72]	3.14 [3.03, 3.25]	-41.2	different
p95 latency, +Grafana (ms)	5.51 [5.09, 5.92]	3.03 [2.97, 3.08]	-45.1	different
App CPU%, Docker	11.02 [9.47, 12.56]	7.42 [6.37, 8.46]	-32.7	different
App CPU%, +Prometheus	3.76 [1.64, 5.88]	3.50 [1.81, 5.18]	-7.1	consistent
App CPU%, +Grafana	1.79 [0.29, 3.28]	1.91 [0.65, 3.16]	$+6.6$	consistent
App mem, Docker (MB)	36.11 [35.91, 36.31]	37.66 [37.49, 37.82]	$+4.3$	different
App mem, +Prometheus (MB)	35.68 [35.47, 35.88]	37.74 [37.50, 37.98]	$+5.8$	different
App mem, +Grafana (MB)	35.79 [35.32, 36.25]	37.72 [37.49, 37.95]	$+5.4$	different
cAdvisor CPU%, +Prometheus	6.12 [4.58, 7.67]	2.41 [1.80, 3.02]	-60.7	different
cAdvisor CPU%, +Grafana	6.40 [4.98, 7.82]	2.19 [1.60, 2.78]	-65.8	different
Prometheus CPU%, +Prometheus	0.78 [0.40, 1.16]	0.62 [0.26, 0.98]	-20.4	consistent
Prometheus CPU%, +Grafana	0.80 [0.47, 1.13]	0.51 [0.18, 0.84]	-36.1	consistent
Grafana CPU%, +Grafana	0.06 [0.01, 0.11]	0.05 [0.03, 0.07]	-22.5	consistent
CPU-stress MTTD (s)	5.65 [3.88, 7.42]	2.41 [2.07, 2.75]	-57.3	different
CPU-stress peak CPU%	96.59 [91.79, 101.39]	95.13 [89.69, 100.57]	-1.5	consistent
Time to OOM (s)	—	—	—	no values (0/20)
Peak memory (MB)	135.54 [135.46, 135.62]	138.59 [137.53, 139.65]	$+2.3$	different
Requests before OOM	5.25 [4.73, 5.77]	16.00 [16.00, 16.00]	$+204.8$	different
Failure MTTD (s)	1.15 [1.05, 1.25]	3.14 [3.07, 3.20]	$+172.9$	different
MTTR (s)	1.99 [1.99, 2.00]	1.98 [1.94, 2.02]	-0.7	consistent
Time to OOM, non-zero fill (s)	11.58 [10.51, 12.66]	11.06 [11.06, 11.06]	-4.5	different
Peak memory, non-zero fill (MB)	135.54 [135.46, 135.62]	136.41 [136.25, 136.56]	$+0.6$	different
Requests before OOM, non-zero fill	5.25 [4.73, 5.77]	5.00 [5.00, 5.00]	-4.8	different

not perfect, and the one observed failure was attributable to a transient delay in the trigger step rather than to the pipeline's own robustness. Critically, 87.5% is a point estimate that should not be quoted on its own: the 95% Wilson score interval for 7 successes out of 8 trials is $[52.9\%, 97.8\%]$ — at this sample size the data are consistent with anywhere from roughly half to nearly all CI/CD deployments succeeding, and the point estimate alone overstates the precision of the claim. A larger n for the CI/CD arm (constrained here by real GitHub Actions minutes and wall-clock cost, Section X) would be needed to narrow this interval and bound the reliability claim more tightly.

X. LIMITATIONS

- **Replication on the same hardware did not reproduce most absolute numbers.** Re-running Experiments 1–5 with the unmodified scripts on the same machine after the operating system was reinstalled (CachyOS, kernel 7.2.9, Docker 29.8.2, replacing Arch/Omarchy 7.1.9; performance power profile; 20 runs each) (Table IX) gave deployment times 12–29% faster, throughput about 48% higher, CPU-stress detection of 2.41s rather than 5.65s and failure detection of 3.14s rather than 1.15s, with the mean outside the original 95% interval for most metrics. Only the Docker+CI/CD deployment time (48.6 s vs. 50.2 s; 8/8 real GitHub Actions runs succeeded vs. 7/8 originally), MTTR, peak CPU under stress, the Prometheus/Grafana CPU figures and the application CPU% in the +Prometheus and +Grafana cases stayed consistent; the replication success rate has a 95% Wilson interval of [0.68, 1.00], so it does not contradict the original 7/8. The deployment ordering (manual, then Docker, then full pipeline) was preserved, but the throughput ordering across monitoring cases was not (4041, 3996 and 4033 req/s for Docker, +Prometheus and +Grafana); absolute values are specific to one software environment.
- **The CPU-contention explanation is not supported.** Experiment 2's application CPU% is bimodal because its ≈0.11s load window is sampled by a longer `docker stats` interval, so the reported fall with monitoring is mostly runs that read as idle. A new Experiment 6 (20 runs per case, fixed 20s load, CPU from kernel counters over exactly the load window, randomized case order, optional CPU-pinned arm) found the monitoring containers used ≈0.03 cores in total, and the per-request client showed no throughput difference from Docker-only whose 95% interval excluded zero. That client is unreliable: its throughput varied roughly three-fold, tracked host load ($r = -0.98$ with host busy cores) and collapsed

TABLE X

WHICH FINDINGS SURVIVED REPLICATION, COMPUTED FROM TABLE IX AND THE RAW DATA (`REPRODUCIBILITY_TABLE.PY`). *Reproduces*: EVERY METRIC IN THE GROUP HAS THE ORIGINAL MEAN INSIDE THE REPLICATION'S 95% INTERVAL, OR THE STATED QUALITATIVE RESULT HELD. *Environment-dependent*: EVERY METRIC IN THE GROUP LIES OUTSIDE IT, OR THE QUALITATIVE RESULT CHANGED. *Not supported*: CONTRADICTED BY EXPERIMENT 6. CONSISTENCY IS A WEAK TEST WHERE THE REPLICATION INTERVAL IS WIDE (THE CPU% ROWS), SO "REPRODUCES" THERE MEANS NOT DISTINGUISHABLE, NOT SHOWN EQUAL. PERCENTAGES ARE THE CHANGE FROM THE ORIGINAL MEAN TO THE REPLICATION MEAN.

Finding	Status	Evidence
Deployment ordering: manual $<$ Docker $<$ full pipeline	Reproduces	Order preserved in all three means
Docker+CI/CD deployment time	Reproduces	Change -3.2%
MTTR after container failure	Reproduces	Change -0.7%
Peak CPU under CPU stress	Reproduces	Change -1.5%
Prometheus and Grafana CPU draw	Reproduces	Changes -36.1 to -20.4%
Application CPU% reading falls as monitoring is added (observation only)	Reproduces	Falls in the replication too ($7.4 \rightarrow 3.5 \rightarrow 1.9\%$); the reading itself is a sampling artifact (last rows)
OOM kill when the allocation is resident (non-zero fill)	Reproduces	20/20 killed; requests and peak memory within 1 request and 1 MB of the original
Absolute deployment times	Environment-dependent	Change -28.9 to -11.8%
Throughput level	Environment-dependent	Change $+47.6$ to $+64.7\%$
Throughput falls as monitoring is added	Environment-dependent	Replication means 4041, 3996, 4033 req/s: not monotone
p95 latency	Environment-dependent	Change -45.1 to -39.5%
Application memory	Environment-dependent	Change $+4.3$ to $+5.8\%$
cAdvisor CPU draw	Environment-dependent	Change -65.8 to -60.7%
CPU-stress detection time	Environment-dependent	Change -57.3%
Failure detection time	Environment-dependent	Change $+172.9\%$
OOM kill with zero-filled allocation	Environment-dependent	20/20 killed originally, 0/20 in the replication (THP shrinker)
Host CPU contention explains the throughput drop	Not supported	Monitoring containers 0.03 cores; keep-alive throughput within 1.4% of Docker-only
Application CPU% falls because of contention	Not supported	Reading is bimodal (0.11 s load window sampled by a longer interval), not a CPU effect

as TIME_WAIT sockets exhausted the ephemeral ports (Section VIII-G), so its pinning result (difference-in-differences $+376$ req/s, 95% CI $[-343, +1061]$) is inconclusive. A keep-alive client is stable and bounds the monitoring cost at about 1.4% of throughput, but one of its two configurations (four load-generator processes, 10 runs per case, no multiple-comparison correction) detects a loss of that size whose mechanism we did not identify. The experiments can rule out large monitoring overhead on this host; they cannot rule out small effects.

- **What is and is not reproducible.** From the committed raw data, `make reproduce` rebuilds every table, statistic and figure in the paper without Docker: unit tests, summary tables checked against the committed copies, every number quoted for Experiments 1–5 and 6 found verbatim in the paper, confidence intervals and effect sizes, and every plotted figure compared pixel for pixel. Two things are outside that guarantee. First, re-running the experiments (`make campaign`) collects new measurements, and the replication shows these differ with the software environment, so a rerun reproduces the procedure, not the numbers. Second, Fig. 3 is a screenshot of a live dashboard and differs on every capture; only the underlying Prometheus series are saved. The original 2026-09 host (kernel, THP setting) no longer exists, so the original runs themselves cannot be re-executed under their original conditions.
- **Experiment 4 depends on kernel behaviour** (mechanism and evidence in Section VIII-E). The retained allocation is zero-filled. On the replication kernel, which enables transparent hugepage shrinking of zero-filled pages (`shrink_underused=1`), the kernel reclaimed that memory under cgroup pressure, so 0/20 containers were OOM-killed even though 16 requests ($\approx$320 MB) succeeded against a 150 MB limit. With the allocation writing a non-zero byte to every page (`/memory?fill=true`, default off), 20/20 runs were OOM-killed after a mean 5.0 requests with peak memory 136.4 MB, close to the original result (5.25 requests, 135.5 MB; the interval rule of Table IX marks these rows different because the replication runs are nearly identical). The OOM finding therefore holds when the memory is genuinely resident; the original zero-filled variant is kernel dependent. The original run was on a different distribution and kernel (Arch 7.1.9 vs. CachyOS 7.2.9); the replication host has transparent hugepages set to `always`. We did not record the original host's THP setting and the original installation no longer exists, so we cannot confirm that this setting is what differed; the distribution default being `madvise` is a plausible but untested explanation.
- **cAdvisor container-label resolution.** This host's Docker

uses the containerd-snapshotter storage driver rather than classic overlay2, which cAdvisor v0.49.1 cannot resolve container name/image labels against. CPU-stress and failure-detection experiments therefore use the application's own native Prometheus metrics rather than cAdvisor's per-container series; Table VII's per-container figures come from `docker stats` directly. This is an environment/tooling limitation rather than a consequence of the experimental design.

- **GHCR authentication.** The Docker+CI/CD deployment step could not pull from GitHub Container Registry (the CLI token used lacks the `read:packages` scope, which would require an interactive browser reauthentication out of scope for an unattended experiment run) and instead deploys the locally built image from the same commit and Dockerfile CI just built. The CI timing itself (test + build + push) is unaffected and fully real.
- **Reduced n for the CI/CD arm.** Only 8 (vs. 20 for the other three deployment configurations) real GitHub Actions runs were triggered, because each run costs ≈45–65s of real CI wall time and consumes Actions minutes; this narrows the confidence interval on the CI/CD reliability and timing figures relative to the other three configurations.
- **Single-host measurement.** All experiments ran on one physical machine with no isolation from other host activity beyond what Docker itself provides; absolute timing figures (especially the sub-2s deployment times) should be read as relative comparisons between configurations on this host, not as portable absolute benchmarks.
- **Related work is still not a systematic review.** Section II's four tabulated papers came from a small number of targeted IEEE Xplore queries; four further works (monitoring overhead, CI usage, delivery performance, container background) were added from targeted web searches, and three more candidates are listed but not reviewed in depth. No multi-database protocol (ACM, Springer, USENIX, inclusion criteria, screening) was followed, so the research-gap claim — that few studies combine all three pillars with controlled quantitative baselines — rests on this limited search and should be rechecked against a systematic review before submission.
- **Detection thresholds are fixed, not tuned.** The 60% CPU-percent threshold (Experiment 3) and the memory container limit (150 MB, Experiment 4) were chosen to make the induced condition reliably observable within a short polling window rather than to model a specific production SLO; detection-time figures should be read relative to these thresholds, not as threshold-independent constants.
- **Not every reported difference is statistically significant.** Welch's t-tests on the raw per-run data (`statistical_analysis.py`) confirm the Docker-to-+Grafana endpoints of the throughput and CPU% trends in Section VIII ($p<0.01$ for both), but two things reported only as directional observations do not reach significance at $\alpha=0.05$: the latency increase across all three monitoring cases ($p \geq 0.36$ for every pair), and the middle step of the throughput trend (Docker vs. +Prometheus, $p=0.177$). Means and standard deviations alone, without this check, would have overstated how uniformly these effects hold run-to-run.

XI. Conclusion

This study built a small but complete containerized CI/CD pipeline with integrated Prometheus–Grafana observability and evaluated it experimentally against a non-containerized baseline and against itself at increasing monitoring depth, across five controlled experiments totaling 188 runs plus a validation experiment on monitoring overhead. The headline findings are that containerization's own deployment-time and resource cost is small and consistent with prior literature, that CI/CD automation's real cost is the repeated build-test cycle rather than the deployment step itself, that Prometheus reliably detects both induced resource saturation and induced failure within low-single-digit seconds bounded by its scrape interval, and that the dominant cost of running a monitoring stack alongside the monitored application on constrained hardware was not established: the original run suggested host-level CPU contention, but the replication and a controlled follow-up experiment (Section X) did not support it and could rule out only large overhead (bounded at about 1.4% of throughput). Table X separates the findings that reproduced from those that depend on the environment. The full raw measurement set, pipeline configuration, and analysis scripts underlying every table in this paper are published at https://github.com/vinayaktyagi10/cicd-monitoring-study to support independent verification.

Acknowledgment

The author thanks Dr. Dibakar Sinha for his guidance and supervision throughout this study.

References

[1] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, "An updated performance comparison of virtual machines and Linux containers," in *2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. Philadelphia, PA, USA: IEEE, 2015, pp. 171–172.

[2] A. Lingayat, R. R. Badre, and A. K. Gupta, "Performance evaluation for deploying docker containers on baremetal and virtual machine," in *2018 3rd International Conference on Communication and Electronics Systems (ICCES)*. Coimbatore, India: IEEE, 2018.

[3] I. Mavridis and H. Karatza, "Performance and overhead study of containers running on top of virtual machines," in *2017 IEEE 19th Conference on Business Informatics (CBI)*. Thessaloniki, Greece: IEEE, 2017.

[4] E. M. I. M. Ekanayaka, J. K. K. H. Thathsarani, D. S. Karunanayaka, N. Kuruwitaarachchi, and N. Skandhakumar, "Enhancing DevOps infrastructure for efficient management of microservice applications," in *2023 IEEE International Conference on e-Business Engineering (ICEBE)*. Sydney, Australia: IEEE, 2023.

[5] M. Dinga, I. Malavolta, L. Giamattei, A. Guerriero, and R. Pietrantuono, "An empirical evaluation of the energy and performance overhead of monitoring tools on Docker-based systems," in *Service-Oriented Computing (ICSOC 2023), Part I*, 2023, pp. 181–196.

[6] M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, "Usage, costs, and benefits of continuous integration in open-source projects," in *Proc. 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2016, pp. 426–437.

[7] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.

[8] D. Bernstein, "Containers and cloud: From LXC to Docker to Kubernetes," *IEEE Cloud Computing*, vol. 1, no. 3, pp. 81–84, 2014.

[9] S. Kumar, N. Bala, A. P. Singh, and Y. Raj, "Cloud-native continuous integration/continuous deployment (CI/CD) pipeline," in *2024 Second International Conference on Advanced Computing & Communication Technologies (ICACCTech)*. IEEE, 2024, title, authorship and venue verified via IEEE Xplore search; full text not reviewed at drafting time.

[10] R. Ashtagi, C. Belani, P. Bhalerao, Y. Bhalerao, and A. Chawle, "Building resilient CICD pipelines: A DevOps security-first framework," in *2025 International Conference on Computational, Communication and Information Technology (ICCCIT)*. IEEE, 2025, title, authorship and venue verified via IEEE Xplore search; full text not reviewed at drafting time.

[11] T. Abirami, S. Mapari, P. Jayadharshini, L. Krishnasamy, and R. R. Vigneshwaran, "Streamlined deployment and monitoring of cloud-native applications on AWS with Kubernetes Prometheus Grafana," in *2023 International Conference on Advances in Computation, Communication and Information Technology (ICAICCIT)*. IEEE, 2023, title, authorship and venue verified via IEEE Xplore search; full text not reviewed at drafting time.